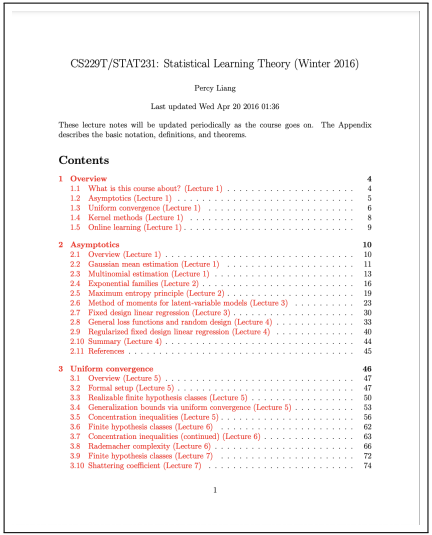
# Trustworthy Machine Learning

## Online Learning

**Sangdon Park**

POSTECH

# Contents from



CS229T/STAT231: Statistical Learning Theory (Winter 2016)

Percy Liang

Last updated Wed Apr 20 2016 01:36

These lecture notes will be updated periodically as the course goes on. The Appendix describes the basic notation, definitions, and theorems.

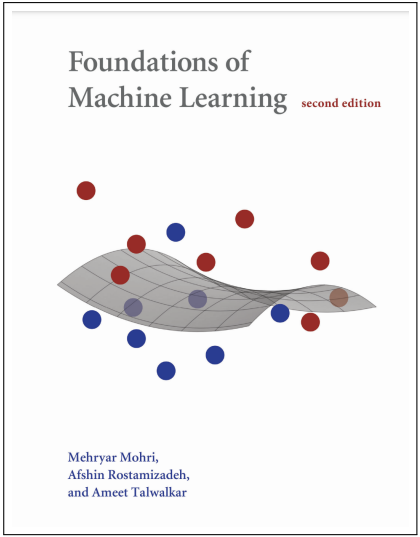
## Contents


**1 Overview** **4**
- 1.1 What is this course about? (Lecture 1) . . . . . 4
- 1.2 Asymptotics (Lecture 1) . . . . . 5
- 1.3 Uniform convergence (Lecture 1) . . . . . 6
- 1.4 Kernel methods (Lecture 1) . . . . . 8
- 1.5 Online learning (Lecture 1) . . . . . 9

**2 Asymptotics** **10**
- 2.1 Overview (Lecture 1) . . . . . 10
- 2.2 Gaussian mean estimation (Lecture 1) . . . . . 11
- 2.3 Multinomial estimation (Lecture 1) . . . . . 13
- 2.4 Exponential families (Lecture 2) . . . . . 16
- 2.5 Maximum entropy principle (Lecture 2) . . . . . 19
- 2.6 Method of moments for latent-variable models (Lecture 3) . . . . . 23
- 2.7 Fixed design linear regression (Lecture 3) . . . . . 30
- 2.8 General loss functions and random design (Lecture 4) . . . . . 33
- 2.9 Regularized fixed design linear regression (Lecture 4) . . . . . 40
- 2.10 Summary (Lecture 4) . . . . . 44
- 2.11 References . . . . . 45

**3 Uniform convergence** **46**
- 3.1 Overview (Lecture 5) . . . . . 47
- 3.2 Formal setup (Lecture 5) . . . . . 47
- 3.3 Realizable finite hypothesis classes (Lecture 5) . . . . . 50
- 3.4 Generalization bounds via uniform convergence (Lecture 5) . . . . . 53
- 3.5 Concentration inequalities (Lecture 5) . . . . . 56
- 3.6 Finite hypothesis classes (Lecture 6) . . . . . 62
- 3.7 Concentration inequalities (continued) (Lecture 6) . . . . . 63
- 3.8 Rademacher complexity (Lecture 6) . . . . . 66
- 3.9 Finite hypothesis classes (Lecture 7) . . . . . 72
- 3.10 Shattering coefficient (Lecture 7) . . . . . 74


1



Foundations of Machine Learning **second edition**

Mehryar Mohri,
Afshin Rostamizadeh,
and Ameet Talwalkar

and various papers.

# Motivation

- We have considered *statistical learning* (*i.e.*, learning under the i.i.d. assumption)

# Motivation

- We have considered *statistical learning* (*i.e.*, learning under the i.i.d. assumption)
- However, this assumption can be broken, *e.g.*, distribution shift, price data

# Motivation

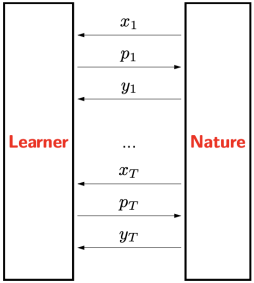
- We have considered *statistical learning* (*i.e.*, learning under the i.i.d. assumption)
- However, this assumption can be broken, *e.g.*, distribution shift, price data
- Here, we will weaken this assumption.
  - batch to online: "how data arrives"
  - statistical to adversarial: "how data are generated"

# Setup

- Prediction task: learn to map an example $x \in \mathcal{X}$ to a label $y \in \mathcal{Y}$

# Setup

- Prediction task: learn to map an example $x \in \mathcal{X}$ to a label $y \in \mathcal{Y}$
- Online learning game between a learner and nature




**Protocol**:

**for** $t = 1, \ldots, T$ **do**

Learner receives an example $x_t \in \mathcal{X}$

Learner outputs prediction $p_t \in \mathcal{Y}$

Learner receives a true label $y_t \in \mathcal{Y}$

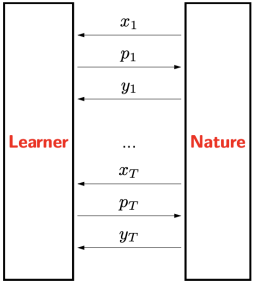
Learner suffers loss $\ell(y_t, p_t)$

Learner update model parameters

**end for**

# Setup

- Prediction task: learn to map an example $x \in \mathcal{X}$ to a label $y \in \mathcal{Y}$
- Online learning game between a learner and nature




**Protocol**:

**for** $t = 1, \ldots, T$ **do**

Learner receives an example $x_t \in \mathcal{X}$

Learner outputs prediction $p_t \in \mathcal{Y}$

Learner receives a true label $y_t \in \mathcal{Y}$

Learner suffers loss $\ell(y_t, p_t)$

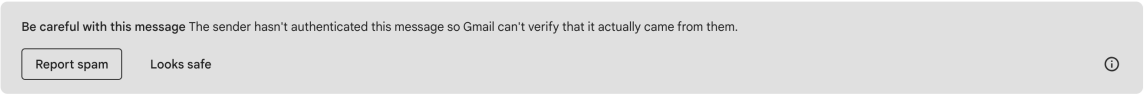
Learner update model parameters

**end for**

- The learner is a function $\mathcal{A}$ that returns the current prediction given the full history, *i.e.*,

$$p_{t+1} = \mathcal{A}(x_{1:t}, p_{1:t}, y_{1:t}, x_{t+1})$$

# Example: Online Binary Classification for Spam Filtering




- examples: $\mathcal{X} := \{0,1\}^d$ are boolean feature vectors (presence or absence of a word)
- labels: $\mathcal{Y} := \{+1, -1\}$ are whether a document is spam or not
- zero-one loss: $\ell(y_t, p_t) = \mathbb{1}\left(y_t \neq p_t\right)$ is whether the prediction was incorrect

# Remarks

- In batch learning, we have a training phase and test phase; but in online learning, they are interleaved.

# Remarks

- In batch learning, we have a training phase and test phase; but in online learning, they are interleaved.
- The online learning setup allows to use the full history.
  - The history grows in time by $O(T)$ and in space by $O(T)$.
  - This means that we can use any batch learning algorithm on the history at each time.
  - However, online algorithms tend to be lightweight, *i.e.,* the amount of work by an algorithm should not grow with $t$.

# Remarks

- In batch learning, we have a training phase and test phase; but in online learning, they are interleaved.
- The online learning setup allows to use the full history.
  - The history grows in time by $O(T)$ and in space by $O(T)$.
  - This means that we can use any batch learning algorithm on the history at each time.
  - However, online algorithms tend to be lightweight, *i.e.*, the amount of work by an algorithm should not grow with $t$.
- Online learning algorithms have the potential to adapt.
  - *e.g.*, we have labels on adversarial examples!

# Remarks

- In batch learning, we have a training phase and test phase; but in online learning, they are interleaved.
- The online learning setup allows to use the full history.
  - The history grows in time by $O(T)$ and in space by $O(T)$.
  - This means that we can use any batch learning algorithm on the history at each time.
  - However, online algorithms tend to be lightweight, *i.e.*, the amount of work by an algorithm should not grow with $t$.
- Online learning algorithms have the potential to adapt.
  - *e.g.*, we have labels on adversarial examples!
- For some applications (*e.g.*, spam filtering), examples are generated by an adversary.

# “Goodness” Metric

How to measure the performance of an online learning algorithm?

# “Goodness” Metric

How to measure the performance of an online learning algorithm?

- The cumulative loss of the learner, *i.e.*,

$$\sum_{t=1}^{T} \ell(y_t, p_t)$$

# "Goodness" Metric

How to measure the performance of an online learning algorithm?

- The cumulative loss of the learner, *i.e.*,

$$\sum_{t=1}^{T} \ell(y_t, p_t)$$

- No! In the adversarial setting, the adversary can manipulate data to make the learner trivially bad loss.

# "Goodness" Metric

How to measure the performance of an online learning algorithm?

- The cumulative loss of the learner, *i.e.*,

$$\sum_{t=1}^{T} \ell(y_t, p_t)$$

- No! In the adversarial setting, the adversary can manipulate data to make the learner trivially bad loss.
- What do you do when your grade is awful? Compare to the best grade in your class!

# Regret

## Definition

$$\text{Regret}(h) := \underbrace{\sum_{t=1}^{T} \ell(y_t, p_t)}_{\text{learner}} - \underbrace{\min_{h \in \mathcal{H}} \sum_{t=1}^{T} \ell(y_t, h(x_t))}_{\text{best expert}}$$

- $\mathcal{H}$ is a class of experts.
- The best export is a role model of the leaner.
- We will consider the worst case regret (*i.e.*, labeled examples are generated by an adversary)

# Need Randomization

**Setup**:

- binary classification, *i.e.*, $y \in \{-1, +1\}$
- zero-one loss, *i.e.*, $\ell(y_t, p_t) := \mathbb{1}(p_t \neq y_t)$
- the learner is fully **deterministic**.

**claim**

*For all learner $\mathcal{A}$, there exists an $\mathcal{H}$ and the sequence of labeled examples such that*

$$\textit{Regret} \geq \frac{T}{2}.$$

- Too bad...
- Why?

# Need Randomization: Why? Intuition

- An adversary (who has full knowledge of the learner) can choose $y_t$ to make it different to the learner's choice $p_t$.
- Thus, the learner's cumulative loss is $T$!
- Not yet; how about the best expert's loss?
- Consider two experts, *i.e.*, $\mathcal{H} := \{h_{-1}, h_{+1}\}$ (where $h_y$ always predict $y$).
- Thus, we have

$$\ell(y_t, h_{-1}(x_t)) + \ell(y_t, h_{+1}(x_t)) = 1 \quad \Rightarrow \quad \sum_{t=1}^{T} \ell(y_t, h_{-1}(x_t)) + \sum_{t=1}^{T} \ell(y_t, h_{+1}(x_t)) = T$$

$$\Rightarrow \sum_{t=1}^{T} \ell(y_t, h_{-1}(x_t)) \leq \frac{T}{2} \quad \text{or} \quad \sum_{t=1}^{T} \ell(y_t, h_{+1}(x_t)) \leq \frac{T}{2}$$

$$\Rightarrow \text{Regret} := \underbrace{\sum_{t=1}^{T} \ell(y_t, p_t)}_{=T} - \underbrace{\min_{h \in \mathcal{H}} \sum_{t=1}^{T} \ell(y_t, h(x_t))}_{\leq \frac{T}{2}} \geq \frac{T}{2}.$$

# Outline

- Having Algorithm
  - Deterministic
  - Realizable assumption
  - Finite $\mathcal{H}$
- Exponential Weighting Algorithm
  - Randomized
  - No realizable assumption
  - Finite $\mathcal{H}$
- Perceptron Algorithm
  - Deterministic
  - Realizable assumption
  - Infinite $\mathcal{H}$

# Add an Assumption without Randomization

**Assumption (realizable)**

*Assume that the best expert $h^* \in \mathcal{H}$ obtains zero cumulative loss (i.e., $\ell(y_t, h^*(x_t) = 0$ for all $t \in \{1, \ldots, T\}$).*

- This impose restrictions on adversaries.
- We saw a similar assumption in PAC learning.
- Practical setup? adaptive conformal prediction

# Halving Algorithm

**Idea**:

- Combine opinions of experts.
- Under the realizable assumption, keep only correct experts.

---

```
HALVING(H)
  H_1 ← H
  for t ← 1 to T do
      RECEIVE(x_t)
      ŷ_t ← MAJORITYVOTE(H_t, x_t)
      RECEIVE(y_t)
      if (ŷ_t ≠ y_t) then
          H_{t+1} ← {c ∈ H_t : c(x_t) = y_t}
      else H_{t+1} ← H_t
  return H_{T+1}
```

---

- $\mathcal{H}_t$: a set of correct experts.

# Halving Algorithm: A Regret Bound

**Theorem**

*Under the realizable assumption, for any* $((x_t, y_t))_{t=1}^{T}$*, we have*

$$\textit{Regret} \leq \log_2 |\mathcal{H}|.$$

# Halving Algorithm: A Regret Bound

**Theorem**

*Under the realizable assumption, for any* $((x_t, y_t))_{t=1}^T$*, we have*

$$\textit{Regret} \leq \log_2 |\mathcal{H}|.$$

**Proof sketch**:

- Let $M$ be the number of mistakes.
- For each mistake, at least half of the experts are eliminated, *i.e.,* if $p_i$ made a mistake,

$$\frac{V_{i+1}}{V_i} \leq \frac{1}{2} \Rightarrow \frac{|V_{T+1}|}{|\mathcal{H}|} \leq \frac{1}{2^M}.$$

- Due to the realizable assumption, we have

$$1 \leq |V_{T+1}|.$$

- $M =$ Regret.

# Halving Algorithm: A Regret Bound

**claim**

*Under the realizable assumption, for any* $((x_t, y_t))_{t=1}^{T}$*, we have*

$$\textit{Regret} \leq \log_2 |\mathcal{H}|.$$

- Very strong results due to the realizable assumption.
  - after a finite number of iterations, the predictor never makes mistakes.

# Remove the Realizable Assumption

- The realizable assumption is too strong
- Let remove this.
- Then, we need a randomization algorithm.
- One example: Exponential weighing algorithm.

# Exponential Weighing Algorithm

- $\mathcal{H}$: a set of experts
- $\ell(\cdot) \in [0, 1]$

**Algorithm 1** Exponential Weighting Algorithm

```
w_1 ← (1/|H|, ..., 1/|H|)
for t = 1, ..., T do
    Observe x_t
    Predict ŷ_t = h^{i_t}(x_t), where i_t ~ w_t
    Observe y_t
    Update w_{t+1}(i) ∝ w_t(i) exp{−η ℓ(h^i(x_t), y_t)} for all i ∈ {1, ..., |H|}
end for
```

# Exponential Weighing Algorithm: A Regret Bound

**Theorem**

*For any loss function $\ell$ with the range in $[0,1]$, if $\eta = \frac{8\ln|\mathcal{H}|}{T}$, we have*

$$\mathbb{E}\textit{Regret} \leq \sqrt{T\ln|\mathcal{H}|}$$

- No realizable assumption.
- "learnable", *i.e.*, $\frac{\text{Regret}}{T} = \sqrt{\frac{\ln|\mathcal{H}|}{T}}$ with a mild assumption on loss.
- Still we assume a finite set of experts $\mathcal{H}$.

# Exponential Weighing Algorithm: A Regret Bound I

## Proof sketch

- Let $L_t^i := \sum_{s=1}^t \ell(h_i(x_s), y_s)$ and $W_t := \sum_{i=1}^{|\mathcal{H}|} \exp\{-\eta L_t^i\}$ with $W_0 = |\mathcal{H}|$.
- The lower bound of the "weight difference":

$$\ln \frac{W_T}{W_0} = \ln \sum_{i=1}^{|\mathcal{H}|} \exp\{-\eta L_t^i\} - \ln |\mathcal{H}| \geq \ln \left( \max_{i \in \{1,\ldots,|\mathcal{H}|\}} \exp\{-\eta L_t^i\} \right) - \ln |\mathcal{H}| = -\eta \min_{i \in \{1,\ldots,|\mathcal{H}|\}} L_t^i - \ln |\mathcal{H}|.$$

- The upper bound of the "weight difference":

$$\begin{aligned}
\ln \frac{W_t}{W_{t-1}} &= \ln \frac{\sum_{i=1}^{|\mathcal{H}|} \exp\{-\eta L_t^i\}}{\sum_{i=1}^{|\mathcal{H}|} \exp\{-\eta L_{t-1}^i\}} = \ln \frac{\sum_{i=1}^{|\mathcal{H}|} \exp\{-\eta \ell(h_t^i(x_t), y_t)\} \exp\{-\eta L_{t-1}^i\}}{\sum_{i=1}^{|\mathcal{H}|} \exp\{-\eta L_{t-1}^i\}} \\
&= \ln \mathbb{E}_{i_t \sim w_t} \exp\left\{-\eta \ell(h^{i_t}(x_t), y_t)\right\} \leq -\eta \mathbb{E}_{i_t \sim w_t} \ell(h^{i_t}(x_t), y_t) + \frac{\eta^2}{8} \\
\Rightarrow \ln \frac{W_T}{W_0} &\leq -\eta \sum_{t=1}^T \mathbb{E}_{i_t \sim w_t} \ell(h^{i_t}(x_t), y_t) + \frac{\eta^2 T}{8}
\end{aligned}$$

# Exponential Weighing Algorithm: A Regret Bound II

## Proof sketch

- For any $s \in \mathbb{R}$ and a random variable $X \in [a, b]$, $\ln \mathbb{E} e^{sX} \leq s\mathbb{E}X + \frac{s^2(b-a)^2}{8}$.

- Combine the lower and upper bounds:

$$-\eta \min_{i \in \{1,\ldots,|\mathcal{H}|\}} L_T^i - \ln |\mathcal{H}| \leq -\eta \sum_{t=1}^{T} \mathbb{E}_{i_t \sim w_t} \ell(h^{i_t}(x_t), y_t) + \frac{\eta^2 T}{8} \Rightarrow$$

$$\sum_{t=1}^{T} \mathbb{E}_{i_t \sim w_t} \ell(h^{i_t}(x_t), y_t) + \min_{i \in \{1,\ldots,|\mathcal{H}|\}} L_T^i \leq \frac{\eta T}{8} + \frac{\ln |\mathcal{H}|}{\eta}$$

# Algorithms So Far

- Halving Algorithm
  - Assumption: realizable
  - Assumption: finite $\mathcal{H}$

# Algorithms So Far

- Halving Algorithm
  - Assumption: realizable
  - Assumption: finite $\mathcal{H}$
- Exponential Weighting Algorithm
  - Assumption: finite $\mathcal{H}$

# Algorithms So Far

- Halving Algorithm
  - Assumption: realizable
  - Assumption: finite $\mathcal{H}$
- Exponential Weighting Algorithm
  - Assumption: finite $\mathcal{H}$
- What's the next?

# Algorithms So Far

- Halving Algorithm
  - Assumption: realizable
  - Assumption: finite $\mathcal{H}$
- Exponential Weighting Algorithm
  - Assumption: finite $\mathcal{H}$
- What's the next?
  - Remove the assumption on the finiteness of $\mathcal{H}$ (under some assumptions)

# Algorithms So Far

- Halving Algorithm
  - Assumption: realizable
  - Assumption: finite $\mathcal{H}$
- Exponential Weighting Algorithm
  - Assumption: finite $\mathcal{H}$
- What's the next?
  - Remove the assumption on the finiteness of $\mathcal{H}$ (under some assumptions)
  - Assumption: realizable

# Perceptron Algorithm: History

# Perceptron Algorithm: Setup

- $\mathcal{D}$: change over time but require the realizable assumption.
- $\mathcal{H}$: linear functions – additional assumption
- $\ell$: 0-1 loss for classification

# Perceptron Algorithm

**Algorithm 2** Perceptron Algorithm

```
w_1 ← w_0 := 0
for t = 1, ..., T do
    Receives an example x_t ∈ 𝒳
    ŷ_t ← sign(w_t · x_t)
    Receives a true label y_t ∈ 𝒴
    if ŷ_t ≠ y_t then
        w_{t+1} ← w_t + y_t x_t
    else
        w_{t+1} ← w_t
    end if
end for
```

# Perceptron Algorithm: A Regret Bound

**Theorem**

*Suppose* $\|x_t\| \leq r$ *for all* $t$ *and for some* $r$ *and there exists* $\gamma > 0$ *and* $v \in \mathbb{R}^d$ *such that*

$$\gamma \leq \frac{y_t(v \cdot x_t)}{\|v\|}.$$

*Then, we have*

$$\textit{Regret} \leq \frac{r^2}{\gamma^2}.$$

# Perceptron Algorithm: A Proof Sketch

- Let $\mathcal{J} \subseteq \{1, \ldots, T\}$ be the set of time indices when updated.
- Thus, Regret $= |\mathcal{J}|$.
- From the "margin" assumption,

$$\begin{aligned}
\gamma\text{Regret} &\leq \frac{\sum_{t\in\mathcal{J}} y_t(v \cdot x_t)}{\|v\|} = \frac{v \cdot \sum_{t\in\mathcal{J}} y_t x_t}{\|v\|} \\
&\leq \left\| \sum_{t\in\mathcal{J}} y_t x_t \right\| \qquad (1) \\
&= \left\| \sum_{t\in\mathcal{J}} w_{t+1} - w_t \right\| = \|w_{T+1}\| \\
&= \sqrt{\sum_{t\in\mathcal{J}} \|w_{t+1}\|^2 - \|w_t\|^2} = \sqrt{\sum_{t\in\mathcal{J}} \|w_t + y_t x_t\|^2 - \|w_t\|^2} = \sqrt{\sum_{t\in\mathcal{J}} 2y_t w_t \cdot x_t + \|x_t\|^2} \\
&\leq \sqrt{\sum_{t\in\mathcal{J}} \|x_t\|^2} \leq r\sqrt{\text{Regret}}.
\end{aligned}$$

  - (1): Cauchy-Schwarz inequality, *i.e.*, $u \cdot v \leq \|u\|\|v\|$